

PRED-CRIM

MANUAL TECNICO Y DE ARQUITECTURA

Plataforma web de analisis preventivo del delito

Documento	Manual tecnico
Version del documento	1.1 corregida
Version del proyecto	v0.1.1 - commit e72d080
Fecha de revision	22 de junio de 2026

Facultad de Ingenieria - Universidad Andres Bello

1. Objetivo y alcance

Este documento describe la implementacion real de PRED-CRIM en la version v0.1.1. La fuente de verdad es el repositorio local y su esquema Prisma. El sistema centraliza incidentes historicos, denuncias ciudadanas, configuraciones de analisis, modelos de riesgo, mapas de calor, reportes estadisticos y alertas operativas.

Aclaracion de alcance: La implementacion actual no utiliza Leaflet, PostGIS ni un motor C++. Usa Mapbox GL JS, coordenadas Float en PostgreSQL y un algoritmo heuristico TypeScript basado en grillas.

2. Stack tecnologico verificado

Capa	Tecnologia	Version / uso
Frontend	Next.js + React	Next.js 14.2.18, React 18.3.1, App Router
Lenguaje	TypeScript	TypeScript 5.6.3
Estilos	Tailwind CSS	Tailwind 3.4.15
Mapas	Mapbox GL JS	Mapbox GL 3.24.0; heatmap y seleccion geografica
Backend	Route Handlers de Next.js	Rutas bajo app/api
Autenticacion	NextAuth v5	Credentials, sesiones JWT y bcryptjs
Persistencia	PostgreSQL + Prisma	Prisma 5.22.0; Supabase o PostgreSQL local
Validacion	Zod	Validacion de formularios, payloads y CSV
CSV	Papa Parse	Normalizacion y validacion fila por fila
Graficos	Recharts / componentes propios	Indicadores y distribuciones

3. Arquitectura logica

3.1 Presentacion

Las paginas se implementan con el App Router de Next.js. Los Server Components consultan Prisma y aplican control de roles; los Client Components administran formularios, filtros, mapas y acciones interactivas. Tailwind CSS define la interfaz y lucide-react provee iconos.

3.2 Aplicacion y API

Los Route Handlers bajo app/api reciben solicitudes HTTP, validan sesion y rol, validan los datos con Zod y delegan la persistencia a Prisma. No existe un servicio backend separado.

3.3 Persistencia

Prisma se conecta a PostgreSQL mediante DATABASE_URL para ejecucion y DIRECT_URL para tareas de esquema. Supabase es el proveedor remoto previsto, pero el mismo esquema funciona con PostgreSQL local. Las coordenadas se almacenan como latitude y longitude de tipo Float.

3.4 Servicios externos

- Mapbox GL JS renderiza mapas, marcadores y capas de calor mediante NEXT_PUBLIC_MAPBOX_TOKEN.
- Nominatim/OpenStreetMap se utiliza para geocodificacion inversa desde la ruta /api/geocode/reverse.
- Supabase puede alojar PostgreSQL, pero no es obligatorio para el desarrollo local.

4. Modulos y rutas de interfaz

Ruta	Modulo	Acceso
/	Portada operativa	Publico
/denunciar	Ingreso de denuncia ciudadana con ubicacion	Publico
/login y /register	Autenticacion y registro	Publico
/dashboard	Resumen operativo	Usuarios autenticados segun rol
/datos	Carga CSV e historial de las ultimas 10 cargas	Analista de Datos / Admin
/denuncias	Revision de denuncias	Analista de Datos / Admin
/configuracion	Parametros, pesos y umbrales	Analista de Seguridad / Admin
/modelos	Entrenamiento, historial y publicacion	Analista de Seguridad / Admin
/mapa	Mapa de calor y filtros	Seguridad, Jefatura / Admin
/reportes	Estadisticas y exportacion	Seguridad, Jefatura / Admin
/alertas	Consulta y gestion de alertas	Seguridad, Jefatura, Campo / Admin

5. Modelo de datos

Entidad	Responsabilidad principal
User	Usuario, hash de contrasena, rol, unidad y ultimo acceso.
CrimeCategory	Catalogo delictual con codigo y severidad de 1 a 5.
Incident	Hecho historico o validado, fecha, categoria, coordenadas y zona.
IncidentUpload	Trazabilidad de archivos CSV, filas aceptadas y rechazadas.
CitizenReport	Denuncia publica o autenticada y su estado de revision.
AnalysisConfig	Rango temporal, zona, tamano de grilla, umbral y decay.
ConfigCategoryWeight	Peso de cada categoria para una configuracion.
PredictiveModel	Ejecucion de entrenamiento, metricas, estado y publicacion.
Prediction	Celda geografica, riesgo normalizado y bloque horario.
Alert / AlertDelivery	Alerta generada y asignacion a Jefatura o Personal de Campo.
StatReport	Metadatos y payload de reportes estadisticos persistidos.

Identificadores: Las entidades usan cuid(), no uuid(). Incident relaciona una categoria mediante categoryId y PredictiveModel usa el campo accuracy, no precision.

6. Motor de analisis de riesgo

El modulo lib/predictor.ts implementa una heuristica de densidad espacio-temporal. No es un modelo de aprendizaje automatico ni una prediccion causal de delitos futuros.

1. Seleccionar incidentes dentro del rango temporal y filtro de zona de AnalysisConfig.
2. Ajustar cada coordenada a una celda segun gridSize.
3. Calcular la contribucion: peso de categoria por severidad por factor de recencia.
4. Aplicar decay temporal $\exp(-k * \text{dias}/30)$, donde k es timeDecayFactor.
5. Agrupar contribuciones por celda, categoria y bloques horarios de tres horas.
6. Normalizar respecto de la celda con mayor valor para obtener riskScore entre 0 y 1.
7. Guardar predicciones globales y por franja; calcular metricas descriptivas.

Metrica accuracy: La accuracy almacenada es un indicador sintetico basado en volumen de muestras. No representa precision estadistica validada contra un conjunto de prueba.

7. API implementada

Metodo	Ruta	Funcion
GET/POST	/api/auth/[...nextauth]	Endpoints internos de NextAuth.
POST	/api/users/register	Registro de usuario ciudadano.
POST / GET	/api/reports	Crear denuncia y consultar las ultimas denuncias.
PATCH	/api/reports/[id]	Validar, rechazar o marcar duplicada una denuncia.
POST	/api/incidents/upload	Validar e importar CSV; admite importacion parcial.
POST	/api/config	Crear configuracion y pesos del analisis.
POST	/api/config/[id]/activate	Activar una configuracion.
POST	/api/models/train	Crear y ejecutar un modelo de riesgo.
POST	/api/models/[id]/publish	Publicar modelo y generar alertas sobre umbral.
GET	/api/stats/export	Exportar incidentes filtrados como CSV UTF-8.
PATCH	/api/alerts/[id]	Confirmar, resolver o descartar una alerta.
GET	/api/notifications/reports	Consultar notificaciones de denuncias pendientes.
GET	/api/geocode/reverse	Resolver direccion aproximada desde coordenadas.

8. Autenticacion, autorizacion y seguridad

- NextAuth v5 usa proveedor Credentials y estrategia de sesion JWT.
- Las contraseñas se almacenan con bcrypt; nunca se persiste texto plano.
- middleware.ts redirige a /login cuando una ruta protegida no tiene sesion.
- Las rutas de negocio vuelven a comprobar sesion y rol antes de mutar datos.
- Zod valida payloads; el parser CSV valida fechas y rangos de coordenadas.
- Los secretos permanecen en .env, archivo ignorado por Git.

Rol	Modulos principales
ADMIN	Todos los modulos implementados.
ANALISTA_DATOS	Dashboard, datos y denuncias.
ANALISTA_SEGURIDAD	Dashboard, configuracion, modelos, mapa, reportes y alertas.
JEFATURA	Dashboard, mapa, reportes y alertas.
PERSONAL_CAMPO	Dashboard y alertas.
CIUDADANO	Dashboard; la ruta publica /denunciar esta disponible sin sesion.

9. Limitaciones conocidas

- No existe PostGIS ni busqueda espacial SQL nativa; se usan coordenadas Float.
- No existe asignacion u optimizacion automatica de cuadrillas.
- Las alertas no usan WebSocket ni push movil; se consultan desde la aplicacion.
- La tabla de modelos y el historial de cargas muestran los ultimos 10 registros sin paginacion ni ordenacion interactiva.
- No hay una suite automatizada de pruebas en el repositorio al momento de esta revision.
- La ejecucion remota depende de PostgreSQL/Supabase y del token publico de Mapbox.

10. Fuentes de verdad

- package.json: versiones y scripts.
- prisma/schema.prisma: modelo persistente.
- app/api: contratos HTTP implementados.
- middleware.ts y auth.config.ts: reglas de sesion y roles.
- lib/predictor.ts: algoritmo de riesgo.
- README.md y .env.example: configuracion operativa.